

GENIE LOGICIEL

Liste des TPs

- [GENIE LOGICIEL/TP1](#)

DevOps

Définition

- Développement de logiciel
- Accélérer la livraison
- Livraison continue de valeur
- Union entre développeur et opérateurs
- Changement de culture : plan → develop → deliver → operate → plan...
- Evolution rôles et responsabilités des membres de l'équipe
- Culture

Dépendance

Définition

- Code externe au projet dont ce dernier a besoin pour fonctionner
- Ne pas réinventer la roue
- 2 types (au sens de l'entreprise)
 - Externe
 - Interne

Gestionnaire de dépendances

- Centralisation des packages (recherche)
- Délégation installation/maintenance
- Gérer les différentes versions et éviter les incompatibilités entre les packages
- Détecter les vulnérabilités
- Exemples :
 - npm (Node.js)
 - pnpm (Node.js)
 - Yarn (Node.js)
 - Maven (Java)
 - Gradle (Java)
 - NuGet (.NET)
 - pip (Python)
 - Conda (python)
 - Composer (PHP)
 - Cargo (Rust)

Intégration de nouvelles modifications du code

Problèmes

- Base de code source commune stable, fonctionnelle et cohérente
- Bien travailler en équipe (plusieurs contributeurs)
- Code source cohérent (compilation, standards de code, tests) à chaque modification
- Code livrable à n'importe quel moment
- Si modification respecte pas les critères ci-dessus, ne pas l'intégrer

Intégration continue

- Automatiser la compilation, le testing et le packaging du code à chaque changement

Pipeline d'intégration continue

- Étapes :
 - Récupérer dépendances (dotnet restore)
 - Compilation (dotnet build)
 - Tests (dotnet test)
 - Analyse statique (sonar-scanner)
 - Publication du package/artifact (dotnet publish)

- ...

Pipeline CI/CD

Pourquoi une pipeline plutôt qu'un script ?

- Versionner le code de la pipeline
- Gérer les exécutions au sein d'une plateforme (logs/conserver des runs)
- Mécanismes de hooks/triggers
- Gestion des artefacts intégrée
- Gérer l'environnement de build
- Sécuriser l'application (gestion des secrets)

Multiples plateformes mais un fonctionnement commun

Concepts similaires :

- Triggers
- Agents d'exécution
- Découpage en stages, jobs, steps
- Artefacts
- Séparation en sous-scripts
- Demande d'approbation

Agent d'exécution

- Exécute le code de la pipeline
- Permet de paralléliser le travail à faire
- Possibilités de spécifier pour chaque pipeline quel type d'agent choisir

Jobs, stages, steps, tasks...

image moodle

Artefacts

- Représentent un fichier
- Peuvent être consommés au sein d'un pipeline
- Peuvent être publiés au cours de l'exécution d'un pipeline
- Peuvent être versionnés

Feeds - Flux

- Permettent d'exposer différentes versions d'artefacts
- Gestion de la mise à disposition aux consommateurs via des "vues"
- ...moodle

Différentes optimisations

- Mettre en place du cache pour la partie dépendance
- ...moodle

Cache VS Artefacts

- Cache → exécuter le pipeline plus rapidement
 - réutiliser des fichiers entre plusieurs jobs ou plusieurs exécutions de pipelines
 - éviter de télécharger les mêmes dépendances à chaque exécution du pipeline
- Artefacts → sauvegarder des fichiers produits par un job
 - rendre accessible des fichiers produits par exécution job/pipeline
 - peuvent être téléchargés par un utilisateur ou utilisés par des jobs dans un pipeline
 - utile pour stocker des résultats de tests ou le résultat du build d'une application

Exemple utilisation du cache

- *GitLab CI* :

```
job:
  script: npm install
  cache:
    key:
      files:
        - pnpm-lock.yaml
    paths:
      - .pnpm-store/
```

Exemple d'exécution en parallèle

- [GitLab CI](#) :

```
test:
  script: pnpm run test
  parallel:
    matrix:
      - NODE_VERSION: ['20', '22', '24']
```

Exemple d'exécution en parallèle (dépendance)

- [GitLab CI](#) :

```
typecheck:
  script: pnpm run typecheck
lint:
  script: pnpm run lint
build-and-test:
  needs: [typecheck, lint]
  script:
    - pnpm run build
    - pnpm run test
```

Les variables

- Variables prédéfinies
 - GitLab CI → `CI_COMMIT_BRANCH`, `CI_PIPELINE_ID`, `CI_PROJECT_NAME` ...
 - GitHub Actions
 - Contextes : `github.ref`, `github.sha`, `github.actor` ...
 - Var d'environnement : `GITHUB_REF`, `GITHUB_SHA` ...
- Variables définies par l'utilisateur
 - niveau workflow, job...
 - manuellement lors d'une exécution manuelle
 - sur un schedule
 - dans les settings du projet (secrets)

Conditions dans les pipelines

- [GitLab CI](#) :

```
deploy:
  script: ./deploy.sh
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
```

Les triggers

- Mécanisme de déclenchement d'un pipeline

Les types

- Trigger manuel
- Trigger planifié
- Trigger lié à l'activité d'une branche, à une pull request
- Trigger via l'achèvement d'une autre pipeline
- Trigger via un appel API
- [GitLab CI](#) :

```
workflow:
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
```

Livraisons, déploiements et environnements

Comment ?

- Plusieurs environnements de test semblables à la production
- Déploiement automatisés sur les environnements
- Tests automatisés : intégration, performance, charge, bout en bout...
- Tests fonctionnels par des utilisateurs

Pourquoi ?

- Moins de risques d'erreurs/problèmes pour livrer une nouvelle version d'une application
 - Tout le processus est automatisé
 - Changements ont déjà été déployés et testés dans d'autres environnements
 - Fiabilités du process éprouvé car fait souvent
- Gain en temps et en qualité
- Nouvelle version livrable à la demande

Continuous Delivery VS Continuous Deployment

- Continuous Deployment : pas besoins de validation manuelle pour déployer en production

Patterns et stratégies de déploiement

- Blue green deployment
- Feature flags
- Canary release
- A/B testing
- Ring-based deployment

Infrastructure

- Ensemble des équipements matériels
 - Serveurs, bases de données, routeurs, logiciels...
 - Ressources "on promise" VS "cloud"
- Au départ métier distinct du développement applicatif
- Procédures de déploiements = actions manuelles + script

Infrastructure As Code

- Automatisation gestion et mise à disposition de l'infrastructure via du code
- Moins de complexité/erreurs, environnements déployés plus vite et plus souvent
- Approche déclarative ou impérative
- Solutions natives fournies par les cloud Providers et solutions multi clouds

Pulumi

- Support majorité des cloud providers
- Permet écrire code en Ts, Python Go, .NET, Java
- Fonctionne avec IDE, gestionnaire de package, frameworks

Mémo Git

- `git switch -c <nomBranch>` → création branche
- Remotes → dépôts distants centralisés sur lesquels on se branche (par défaut appelé *origin*)
- Version locale
- *Pull* (*fetch* + *merge* ou *rebase*) → je veux chercher cette version (remote) là
- `git remote -v` → liste des remotes
- `git remote add <nomRemote> <urlDepotRemote>` → ajouter une remote
- `git branch -al` → indique sur quelle branche pointe le pointeur
- `git pull <nomRemote> main` → récupérer les modifications du dépôt distant
- `git reset --hard upstream/main`
- `git reflog` → liste les changements de pointeurs sur le dépôt
- `git rebase -i` → réécrire les commit (`-i` pour mode interactif)
- `git push origin <hashDuCommit>:main` → push le commit (par son hash) sur la branche main